Работа 1. Инкапсуляция, конструкторы, деструкторы

Тема: Инкапсуляция. Конструкторы. Деструкторы. Разделение класса на `.h` и `.cpp`.

Цель работы

Научиться проектировать класс, который гарантирует корректность своих данных. Понять, зачем нужны `private`, конструкторы и деструктор. Освоить разделение класса на заголовочный файл и файл реализации.

Предметная область

Класс `Matrix` — матрица целых чисел произвольной размерности.

Часть I. Проектирование класса

Нужно реализовать класс `Matrix` со следующими возможностями.

1. Поля класса – доступ только у методов своего класса.

Стиль названия полей примем такой: имя_ (например, data_)

2. Конструкторы - три конструктора — разные способы создать матрицу:

- **Конструктор по умолчанию** — создаёт матрицу размерности 0×0. Память не выделяется.

```
Matrix m1;    //0×0
```

- **Конструктор с одним параметром** — создаёт единичную квадратную матрицу указанной размерности. Единичная квадратная матрица — это квадратная матрица (число строк равно числу столбцов), у которой элементы главной диагонали равны 1, остальные — 0.

```
Matrix m2(3); //3×3, единичная
```

- **Конструктор с двумя параметрами** — создаёт матрицу `rows × cols`, заполненную нулями.

```
Matrix m3(3,4); // 3×4, все нули
```

Требования к конструкторам:

- После изучение добавьте: в конструкторе класса реализуйте проверку входных параметров. Если переданная размерность отрицательная, конструктор должен сгенерировать исключение `std::invalid_argument` с понятным сообщением об ошибке (например, "Размерность не может быть отрицательной").
- Если размерность нулевая — матрица создаётся пустой, память не выделяется.
- В конструкторе с двумя параметрами память выделяется построчно: сначала массив указателей на строки, потом каждая строка отдельно.
- Элементы инициализируются нулями ***(вопрос: какие для этого есть способы, показать разные варианты)***

3. Деструктор - освобождает память в обратном порядке относительно выделения:

  - Сначала для каждой строки `delete[] data_[i]`.
  - Потом `delete[] data_`.

Деструктор должен корректно работать с пустой матрицей (0×0).

4. Методы - для именования методов используем стиль camelCase

Доступ к элементам:

- `int get(int i, int j) const` — возвращает элемент. Если индексы вне диапазона — бросает исключение `std::out_of_range` (до изучения исключений – выдать на экран сообщение)
- `void set(int i, int j, int value)` — устанавливает элемент. Если индексы вне диапазона — бросает исключение `std::out_of_range` (до изучения исключений – выдать на экран сообщение)

Заполнение:

- `void inputFromKeyboard()` — заполняет матрицу значениями с клавиатуры. Пользователь вводит `rows_ * cols_` чисел.
- `void fillRandom()` — заполняет матрицу случайными числами от 0 до 99.

Вывод:

- `void print() const` — выводит матрицу на экран, построчно.

Вычисление:

- `int sum() const` — возвращает сумму всех элементов матрицы.

Геттеры:

- `int getRows() const` — количество строк.
- `int getCols() const` — количество столбцов.

Обратите внимание: `get, print, sum, getRows, getCols` — все `const`. Эти методы не меняют объект. Компилятор это проверит.

5. Разделение на файлы (после изучения, первый вариант работы можно без разделения файлов)

- `Matrix.h` — объявление класса.
- `Matrix.cpp` — реализация методов.
- `main.cpp` — демонстрационная программа.

В начале `Matrix.h` — `#pragma once`. Не подключайте `<iostream>` в заголовке — это делается в `.cpp`.

Часть II. Демонстрационная программа

В main напишите программу, которая:

1. Создаёт четыре матрицы разными конструкторами:

```
Matrix m1;          // 0×0
Matrix m2(3);       // 3×3, единичная
Matrix m3(3, 4);    // 3×4, нули
Matrix m4(2, 3);    // 2×3, нули
```

2. Выводит на экран M2, M3, M4.
3. Заполняет M2 по формуле: элемент `(i, j)` равен `i * j`. Выводит на экран.

Пример для M2 (3×3):

0 0 0

0 1 2

0 2 4

4. Заполняет M3 случайными числами через `fillRandom()`. Выводит на экран.
5. Заполняет M4 с клавиатуры через `inputFromKeyboard()`. Выводит на экран.
6. Считает сумму элементов M3 через `sum()`. Выводит на экран.
7. Демонстрирует работу исключений (добавить после изучения):

```
try {
    Matrix bad(-1, 5);            // отрицательная размерность
} catch (const std::invalid_argument& e) {
    std::cout << "Caught: " << e.what() << '\n';
}

try {
    m3.get(100, 100);             // выход за границы
} catch (const std::out_of_range& e) {
    std::cout << "Caught: " << e.what() << '\n';
}
```

Часть III. Требования к сдаче

1. **Структура проекта** (можно сначала выполнить в одном файле):

```
lab1/
├── Matrix.h
├── Matrix.cpp
├── main.cpp
└── README.md
```

2. **Компиляция:**

   После изучения - установить флаги `-Wall -Wextra`

   Как пример, в командной строке: g++ -std=c++17 -Wall -Wextra main.cpp Matrix.cpp -o lab1

3. **README.md** — краткое описание: что делает класс (назначение класса и его основные возможности), как запустить программу и что она должна продемонстрировать, - важные особенности реализации (исключения, пустая матрица и т.п.)
4. **Git:** три коммита минимум

Часть IV. Примеры фрагментов кода

Фрагменты не копируйте полностью — они даны как подсказка, а не как готовое решение. Попробуйте сначала написать самостоятельно.

Объявление класса в Matrix.h

```
#pragma once

#include <stdexcept>

class Matrix {
public:
    Matrix();
    explicit Matrix(int size);
    Matrix(int rows, int cols);
    ~Matrix();

    int get(int i, int j) const;
    void set(int i, int j, int value);
    void inputFromKeyboard();
    void fillRandom();
    void print() const;
    int sum() const;
    int getRows() const;
    int getCols() const;

private:
    int** data_;
    int rows_;
    int cols_;
};
```

Обратите внимание:

- `explicit` перед конструктором с одним параметром — запрещает неявное преобразование `Matrix m = 5;`. Оно должно быть явным: `Matrix m(5);`.
- У всех методов, которые не меняют объект, есть `const` в конце.
- Поля в доступе `private`.

Пример 2. Конструктор с двумя параметрами в `Matrix.cpp`

```
#include "Matrix.h"

Matrix::Matrix(int rows, int cols)
    : data_(nullptr), rows_(rows), cols_(cols) {
    if (rows < 0 || cols < 0) {
        throw std::invalid_argument("Matrix dimensions must be non-
negative");
    }
    if (rows == 0 || cols == 0) {
        return;
    }

    data_ = new int*[rows_];
    for (int i = 0; i < rows_; ++i) {
        data_[i] = new int[cols_]{};   // () — инициализация нулями
    }
}
```

Обратите внимание:

- В списке инициализации конструктора обязательно инициализируйте указатель `data_` значением `nullptr`. Это гарантирует, что даже если в теле конструктора произойдёт исключение, указатель не будет содержать невалидный адрес. В деструкторе удаление `nullptr` безопасно и не приводит к ошибке.
- Проверка на отрицательные размеры — до выделения памяти.
- Пустая матрица — если хотя бы одно измерение равно нулю, матрица по смыслу пустая — в ней нет ни одной ячейки, которую нужно хранить. Значит, выделять под неё память не нужно.
- `new int[cols_]{}` — со скобками в конце, без них элементы не инициализируются нулями.

Пример 3. Деструктор

```
Matrix::~Matrix() {
    if (data_ == nullptr) return;

    for (int i = 0; i < rows_; ++i) {
        delete[] data_[i];
    }
    delete[] data_;
}
```

Обратите внимание:

- Проверка на `nullptr` — чтобы деструктор корректно работал для пустой матрицы.
- `delete[]` для каждой строки, потом `delete[]` для массива указателей.
- Порядок важен: сначала строки, потом массив указателей на них.

Часть VI. Вопросы для самопроверки

1. Что произойдёт, если убрать `const` у метода `sum()`?
2. Почему поля класса объявлены `private`, а не `public`?
3. Зачем нужен `explicit` перед конструктором с одним параметром?
4. Что произойдёт, если в деструкторе забыть `delete[]` для строк?
5. Зачем проверять размерность на отрицательность до выделения памяти?
6. Что произойдёт, если в `set` не проверять индексы?
7. Почему `get` и `set` называют «доступ к элементам», а не «сеттеры/геттеры полей»?

Часть VII. Домашнее задание

1. Добавить метод `Matrix transpose() const`, возвращающий транспонированную матрицу.
2. Проверить: что произойдёт, если скопировать объект `Matrix`?

   После пункта 4 части II добавьте:

   ```
   Matrix mm = m3;

   mm.print();
   ```

   Что не нужно делать сейчас: не пишите здесь конструктор копирования и `operator=`. Мы разберём их позже.